Programmation Evenementielle

Le pattern .on() / .emit() dans Node.js

BUT2 S4 - R401 — Prerequis WebSockets

Joseph Azar — Maitre de Conferences — Universite Marie Louis Pasteur — FEMTO-ST

Plan de cette seance

C'est quoi un evenement ? — Du navigateur a Node.js

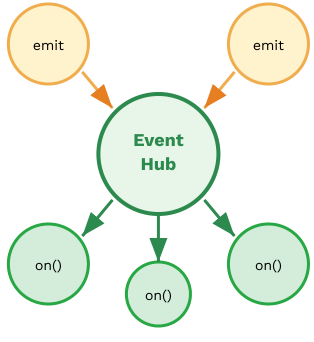
Le trio magique — Emitter, Listener, Handler

EventEmitter — emit() et on() en pratique

Regles du jeu — Ordre, once(), gestion memoire

Exemples concrets — process, http, fs, net

Le lien avec WebSockets — socket.on() et socket.emit()



C'est quoi un evenement ?

Analogie : Imaginez une sonnette de porte.

> **Definition :** Un **evenement** = quelque chose qui se passe. Un **listener** = du code qui attend que ca se passe. Un **handler** = ce qu'on fait quand ca se passe.

Vous connaissez deja les evenements !

En JavaScript dans le navigateur, vous avez deja fait ca :

```
<button id="submit">Valider</button>

<script>
  const btn = document.getElementById('submit');

  btn.addEventListener('click', (event) => {
    console.log('Bouton clique !');
  });
</script>
```

Emitter : l'utilisateur qui clique

Event : `'click'`

Handler : le `console.log`

Bonne nouvelle : Node.js utilise exactement le meme concept, mais cote **serveur** et avec des evenements **personnalises**.

Les 3 acteurs de la programmation evenementielle

C'est le pattern Publish / Subscribe (Pub/Sub)

Retenir : `emit()` = "je crie un message" → `on()` = "j'ecoute ce message" → **callback** = "je reagis"

EventEmitter en Node.js

Le module `events` est **integre** dans Node.js (pas besoin d'installer).

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
```

Exemple complet :

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// 1. D'abord, on ECOUTE (listener + handler)
emitter.on('salut', (data) => {
  console.log('Recu :', data);
});

// 2. Ensuite, on EMET (emitter)
emitter.emit('salut', 'Bonjour le monde !');
```

```
> node app.js
Recu : Bonjour le monde !
```

Vocabulaire : `'salut'` est le **nom de l'evenement** (vous choisissez le nom). `'Bonjour le monde !'` sont les **donnees** transmises.

Regle d'or : L'ordre compte !

Correct : on() AVANT emit()

```
// D'abord ecouter
emitter.on('msg', (d) => {
  console.log(d);
});

// Puis emettre
emitter.emit('msg', 'Hello');
// => "Hello"
```

Faux : emit() AVANT on()

```
// Emettre en premier
emitter.emit('msg', 'Hello');

// Ecouter trop tard !
emitter.on('msg', (d) => {
  console.log(d);
});
// => RIEN (silence)
```

Analogie : Un evenement = une emission TV en direct

Si vous allumez la TV en retard, vous avez rate le debut. Impossible de rembobiner !

Plusieurs listeners sur un meme evenement

Un emit() peut declencher **plusieurs** listeners a la fois :

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Listener 1 : enregistrer dans un log
emitter.on('commande', (item) => {
  console.log('LOG: Commande recue pour', item);
});

// Listener 2 : envoyer un email
emitter.on('commande', (item) => {
  console.log('EMAIL: Confirmation envoyee pour', item);
});

// Listener 3 : mettre a jour le stock
emitter.on('commande', (item) => {
  console.log('STOCK: -1 pour', item);
});

emitter.emit('commande', 'Pizza Margherita');
```

```
> node app.js
LOG: Commande recue pour Pizza Margherita
EMAIL: Confirmation envoyee pour Pizza Margherita
STOCK: -1 pour Pizza Margherita
```

once() vs on() : ecouter une fois ou toujours ?

on() = ecoute indefiniment

```
emitter.on('ping', () => {
  console.log('pong!');
});

emitter.emit('ping'); // pong!
emitter.emit('ping'); // pong!
emitter.emit('ping'); // pong!
```

once() = ecoute 1 seule fois

```
emitter.once('ping', () => {
  console.log('pong!');
});

emitter.emit('ping'); // pong!
emitter.emit('ping'); // (rien)
emitter.emit('ping'); // (rien)
```

Quand utiliser once() ? Pour les evenements qui ne doivent se produire qu'une fois : initialisation, premiere connexion, chargement de config, etc.

Gestion des listeners et memoire

Un listener actif = de la **memoire utilisee**. Si vous oubliez de les supprimer, c'est une **fuite memoire** !

```
// Supprimer UN listener specifique
function monHandler(data) {
  console.log(data);
}

emitter.on('msg', monHandler);
emitter.removeListener('msg', monHandler); // supprime

// Supprimer TOUS les listeners d'un evenement
emitter.removeAllListeners('msg');
```

Limite par defaut : 10 listeners max par evenement. Au-dela, Node.js affiche un warning.

Augmenter la limite :
`emitter.setMaxListeners(50);`

Les evenements sont PARTOUT dans Node.js

Vous utilisez deja des evenements sans le savoir :

process (global) :

```
// Quand on fait CTRL+C
process.on('SIGINT', () => {
  console.log('Arret...');
});

// Erreur non geree
process.on('uncaughtException',
  (err) => {
  console.error(err);
});
```

http (serveur) :

```
const http = require('http');

http.get(url, (res) => {
  // Donnees arrivent par morceaux
  res.on('data', (chunk) => {
    body += chunk;
  });

  // Tout est arrive
  res.on('end', () => {
    console.log(body);
  });
});
```

Encore plus d'evenements Node.js

net (TCP) :

```
const server = require('net')
  .createServer();

server.on('connection',
  (socket) => {
  socket.write('Bienvenue!');

  socket.on('data', (data) => {
    console.log('Recu:', data);
  });

  socket.on('end', () => {
    console.log('Deconnecte');
  });
});

server.listen(3000);
```

fs (fichiers) :

```
const fs = require('fs');

// Ecrire dans un fichier
fs.createWriteStream('log.txt')
  .end('Contenu du log')
  .on('error', () => {
    console.log('Erreur!');
  })
  .on('finish', () => {
    console.log('Ecrit!');
  });
```

Pattern identique :

`objet.on('evenement', callback)`

Toujours le meme duo !

Le lien avec WebSockets / Socket.IO

Socket.IO utilise **exactement le meme pattern** !

EventEmitter (Node.js)

```
// Ecouter
emitter.on('salut', (data) => {
  console.log(data);
});

// Emettre
emitter.emit('salut', 'Hello!');
```

Socket.IO (WebSockets)

```
// Ecouter (serveur)
socket.on('message', (data) => {
  console.log(data);
});

// Emettre (client)
socket.emit('message', 'Hello!');
```

C'est le MEME pattern !

.emit('nom', donnees) → .on('nom', callback)

Si vous comprenez EventEmitter, vous comprenez Socket.IO

Ce qu'on a appris

Evenement = quelque chose qui se passe

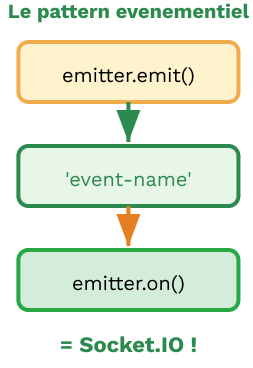
Emitter = celui qui declenche (`.emit()`)

Listener = celui qui ecoute (`.on()`)

Handler = le callback qui reagit

Ordre : `on()` AVANT `emit()` !

Memoire : supprimer les listeners inutiles



Questions ?

Prets pour les WebSockets !